

```
public class User {

    private int userId;
    private String username;
    private String password;
    private String role;

    public User() {
    }

    public User(int userId, String username, String password, String role) {
        this.userId = userId;
        this.username = username;
        this.password = password;
        this.role = role;
    }

    public int getUserId() {
        return userId;
    }

    public void setUserId(int userId) {
        this.userId = userId;
    }

    public String getUsername() {
```

```
        return username;
    }
}
```

```
public void setUsername(String username) {
    this.username = username;
}
```

```
public String getPassword() {
    return password;
}
```

```
public void setPassword(String password) {
    this.password = password;
}
```

```
public String getRole() {
    return role;
}
```

```
public void setRole(String role) {
    this.role = role;
}
```

```
@Override
```

```
public String toString() {
    return "User ID : " + userId +
```

```
        "\nUsername : " + username +  
        "\nRole : " + role;  
    }  
}
```

```
public class Admin extends User {
```

```
    public Admin() {  
        super();  
        setRole("Admin");  
    }
```

```
    public Admin(int userId, String username, String password) {  
        super(userId, username, password, "Admin");  
    }
```

```
    public void displayMenu() {  
        System.out.println("\n===== ADMIN MENU =====");  
        System.out.println("1. Add Location");  
        System.out.println("2. View Locations");  
        System.out.println("3. Approve Event");  
        System.out.println("4. View Pending Events");  
        System.out.println("5. Monitor Seat Capacity");  
        System.out.println("6. Logout");  
    }  
}
```

```
Public class EventManager extends User {
```

```
    Public EventManager() {
```

```
        Super();
```

```
        setRole("Event Manager");
```

```
    }
```

```
    Public EventManager(int userId, String username, String password) {
```

```
        Super(userId, username, password, "Event Manager");
```

```
    }
```

```
    Public void displayMenu() {
```

```
        System.out.println("\n===== EVENT MANAGER MENU =====");
```

```
        System.out.println("1. Create Event");
```

```
        System.out.println("2. Edit Event");
```

```
        System.out.println("3. Send Event for Approval");
```

```
        System.out.println("4. Cancel Event");
```

```
        System.out.println("5. View My Events");
```

```
        System.out.println("6. Logout");
```

```
    }
```

```
    Public void createEvent() {
```

```
        System.out.println("Create Event selected.");
```

```
}
```

```
Public void editEvent() {
```

```
    System.out.println("Edit Event selected.");
```

```
}
```

```
Public void sendForApproval() {
```

```
    System.out.println("Event sent for approval.");
```

```
}
```

```
Public void cancelEvent() {
```

```
    System.out.println("Event cancelled.");
```

```
}
```

```
}
```

```
public class Consumer extends User {
```

```
    public Consumer() {
```

```
        super();
```

```
        setRole("Consumer");
```

```
    }
```

```
    public Consumer(int userId, String username, String password) {
```

```
        super(userId, username, password, "Consumer");  
    }
```

```
    public void displayMenu() {  
        System.out.println("\n===== CONSUMER MENU =====");  
        System.out.println("1. View Approved Events");  
        System.out.println("2. Book Ticket");  
        System.out.println("3. View My Tickets");  
        System.out.println("4. Make Payment");  
        System.out.println("5. Apply Coupon");  
        System.out.println("6. Logout");  
    }
```

```
    public void viewApprovedEvents() {  
        System.out.println("Displaying approved events...");  
    }
```

```
    public void bookTicket() {  
        System.out.println("Ticket booked successfully.");  
    }
```

```
    public void makePayment() {  
        System.out.println("Payment completed successfully.");  
    }
```

```
    public void applyCoupon() {
```

```
        System.out.println("Coupon applied successfully.");
    }
}
```

```
Public class Location {
```

```
    Private int locationId;
```

```
    Private String locationName;
```

```
    Private String address;
```

```
    Private int capacity;
```

```
    Public Location() {
```

```
    }
```

```
    Public Location(int locationId, String locationName, String address, int capacity) {
```

```
        This.locationId = locationId;
```

```
        This.locationName = locationName;
```

```
        This.address = address;
```

```
        This.capacity = capacity;
```

```
    }
```

```
    // Getters
```

```
    Public int getLocationId() {
```

```
    Return locationId;  
}
```

```
Public String getLocationName() {  
    Return locationName;  
}
```

```
Public String getAddress() {  
    Return address;  
}
```

```
Public int getCapacity() {  
    Return capacity;  
}
```

// Setters

```
Public void setLocationId(int locationId) {  
    This.locationId = locationId;  
}
```

```
Public void setLocationName(String locationName) {  
    This.locationName = locationName;  
}
```

```
Public void setAddress(String address) {  
    This.address = address;
```



```
}
```

```
Public void setCapacity(int capacity) {
```

```
    This.capacity = capacity;
```

```
}
```

```
@Override
```

```
Public String toString() {
```

```
    Return "\nLocation ID : " + locationId +
```

```
        "\nLocation Name : " + locationName +
```

```
        "\nAddress : " + address +
```

```
        "\nCapacity : " + capacity;
```

```
}
```

```
}
```

```
public class Event {
```

```
    private int eventId;
```

```
    private String eventName;
```

```
    private String eventDate;
```

```
    private double ticketPrice;
```

```
    private int availableSeats;
```

```
    private Location location;
```

```
    private String status;    // Pending, Approved, Rejected, Cancelled
```

```
    private int managerId;
```

```
public Event() {  
}
```

```
public Event(int eventId, String eventName, String eventDate,  
            double ticketPrice, int availableSeats,  
            Location location, String status, int managerId) {
```

```
    this.eventId = eventId;  
    this.eventName = eventName;  
    this.eventDate = eventDate;  
    this.ticketPrice = ticketPrice;  
    this.availableSeats = availableSeats;  
    this.location = location;  
    this.status = status;  
    this.managerId = managerId;  
}
```

```
// Getters
```

```
public int getEventId() {  
    return eventId;  
}
```

```
public String getEventName() {  
    return eventName;  
}
```

```
public String getEventDate() {  
    return eventDate;  
}
```

```
public double getTicketPrice() {  
    return ticketPrice;  
}
```

```
public int getAvailableSeats() {  
    return availableSeats;  
}
```

```
public Location getLocation() {  
    return location;  
}
```

```
public String getStatus() {  
    return status;  
}
```

```
public int getManagerId() {  
    return managerId;  
}
```

```
// Setters
```

```
public void setEventId(int eventId) {  
    this.eventId = eventId;  
}
```

```
public void setEventName(String eventName) {  
    this.eventName = eventName;  
}
```

```
public void setEventDate(String eventDate) {  
    this.eventDate = eventDate;  
}
```

```
public void setTicketPrice(double ticketPrice) {  
    this.ticketPrice = ticketPrice;  
}
```

```
public void setAvailableSeats(int availableSeats) {  
    this.availableSeats = availableSeats;  
}
```

```
public void setLocation(Location location) {  
    this.location = location;  
}
```

```
public void setStatus(String status) {  
    this.status = status;  
}
```

```
}
```

```
public void setManagerId(int managerId) {  
    this.managerId = managerId;  
}
```

```
@Override
```

```
public String toString() {  
    return "\nEvent ID : " + eventId +  
        "\nEvent Name : " + eventName +  
        "\nEvent Date : " + eventDate +  
        "\nTicket Price : ₹" + ticketPrice +  
        "\nAvailable Seats : " + availableSeats +  
        "\nLocation : " + location.getLocationName() +  
        "\nStatus : " + status +  
        "\nManager ID : " + managerId;  
}  
}
```

```
public class Booking {
```

```
    private int bookingId;
```

```
private int consumerId;

private int eventId;

private int numberOfTickets;

private double totalAmount;

private String paymentStatus; // Pending or Paid


public Booking() {
}


public Booking(int bookingId, int consumerId, int eventId,
               int numberOfTickets, double totalAmount,
               String paymentStatus) {

    this.bookingId = bookingId;
    this.consumerId = consumerId;
    this.eventId = eventId;
    this.numberOfTickets = numberOfTickets;
    this.totalAmount = totalAmount;
    this.paymentStatus = paymentStatus;
}


// Getters

public int getBookingId() {
    return bookingId;
}
```

```
public int getConsumerId() {  
    return consumerId;  
}
```

```
public int getEventId() {  
    return eventId;  
}
```

```
public int getNumberOfTickets() {  
    return numberOfTickets;  
}
```

```
public double getTotalAmount() {  
    return totalAmount;  
}
```

```
public String getPaymentStatus() {  
    return paymentStatus;  
}
```

// Setters

```
public void setBookingId(int bookingId) {  
    this.bookingId = bookingId;  
}
```

```
public void setConsumerId(int consumerId) {
```

```
    this.consumerId = consumerId;
}
```

```
public void setEventId(int eventId) {
    this.eventId = eventId;
}
```

```
public void setNumberOfTickets(int numberOfTickets) {
    this.numberOfTickets = numberOfTickets;
}
```

```
public void setTotalAmount(double totalAmount) {
    this.totalAmount = totalAmount;
}
```

```
public void setPaymentStatus(String paymentStatus) {
    this.paymentStatus = paymentStatus;
}
```

```
@Override
```

```
public String toString() {
    return "\nBooking ID : " + bookingId +
        "\nConsumer ID : " + consumerId +
        "\nEvent ID : " + eventId +
        "\nTickets Booked : " + numberOfTickets +
        "\nTotal Amount : ₹" + totalAmount +
```



```
        "\nPayment Status : " + paymentStatus;
    }
}
```

```
public class Coupon {

    private int couponId;
    private int consumerId;
    private String couponCode;
    private double discountPercentage;
    private boolean used;

    public Coupon() {
    }

    public Coupon(int couponId, int consumerId, String couponCode,
        double discountPercentage, boolean used) {
        this.couponId = couponId;
        this.consumerId = consumerId;
        this.couponCode = couponCode;
        this.discountPercentage = discountPercentage;
        this.used = used;
    }
}
```

```
// Getters
```

```
public int getCouponId() {  
    return couponId;  
}
```

```
public int getConsumerId() {  
    return consumerId;  
}
```

```
public String getCouponCode() {  
    return couponCode;  
}
```

```
public double getDiscountPercentage() {  
    return discountPercentage;  
}
```

```
public boolean isUsed() {  
    return used;  
}
```

```
// Setters
```

```
public void setCouponId(int couponId) {  
    this.couponId = couponId;  
}
```

```
public void setConsumerId(int consumerId) {  
    this.consumerId = consumerId;  
}
```

```
public void setCouponCode(String couponCode) {  
    this.couponCode = couponCode;  
}
```

```
public void setDiscountPercentage(double discountPercentage) {  
    this.discountPercentage = discountPercentage;  
}
```

```
public void setUsed(boolean used) {  
    this.used = used;  
}
```

```
@Override
```

```
public String toString() {  
    return "\nCoupon ID : " + couponId +  
        "\nConsumer ID : " + consumerId +  
        "\nCoupon Code : " + couponCode +  
        "\nDiscount : " + discountPercentage + "%" +  
        "\nStatus : " + (used ? "Used" : "Available");  
}  
}
```

```
import java.util.ArrayList;

public class DataStore {

    public static ArrayList<User> users = new ArrayList<>();
    public static ArrayList<Location> locations = new ArrayList<>();
    public static ArrayList<Event> events = new ArrayList<>();
    public static ArrayList<Booking> bookings = new ArrayList<>();
    public static ArrayList<Coupon> coupons = new ArrayList<>();

    // Static block to initialize default data
    static {

        // Default Admin
        users.add(new Admin(1, "admin", "admin123"));

        // Sample Event Manager
        users.add(new EventManager(2, "manager1", "manager123"));

        // Sample Consumer
        users.add(new Consumer(3, "consumer1", "consumer123"));
```

```
// Sample Location
```

```
locations.add(new Location(  
    101,  
    "Hyderabad Convention Center",  
    "HITEC City, Hyderabad",  
    500));
```

```
// Sample Event
```

```
events.add(new Event(  
    201,  
    "Java Tech Fest",  
    "15-08-2026",  
    499.0,  
    300,  
    locations.get(0),  
    "Approved",  
    2));
```

```
}
```

```
}
```

```
Import java.util.Scanner;
```

```
Public class Authentication {

    Private Scanner sc = new Scanner(System.in);

    // Register User

    Public void register() {

        System.out.println("\n===== USER REGISTRATION =====");

        System.out.print("Enter User ID: ");

        Int userId = sc.nextInt();

        Sc.nextLine();

        System.out.print("Enter Username: ");

        String username = sc.nextLine();

        // Check duplicate username

        For (User user : DataStore.users) {

            If (user.getUsername().equalsIgnoreCase(username)) {

                System.out.println("Username already exists!");

                Return;

            }

        }

        System.out.print("Enter Password: ");

        String password = sc.nextLine();
```

```
System.out.println("\nSelect Role");  
System.out.println("1. Event Manager");  
System.out.println("2. Consumer");  
System.out.print("Enter Choice: ");
```

```
Int choice = sc.nextInt();
```

```
Switch (choice) {
```

```
    Case 1:
```

```
        DataStore.users.add(new EventManager(userId, username, password));  
        System.out.println("Event Manager Registered Successfully.");  
        Break;
```

```
    Case 2:
```

```
        DataStore.users.add(new Consumer(userId, username, password));  
        System.out.println("Consumer Registered Successfully.");  
        Break;
```

```
    Default:
```

```
        System.out.println("Invalid Role.");
```

```
}
```

```
}
```

```
// Login
Public User login() {

    System.out.println("\n===== LOGIN =====");

    Sc.nextLine();

    System.out.print("Username: ");
    String username = sc.nextLine();

    System.out.print("Password: ");
    String password = sc.nextLine();

    For (User user : DataStore.users) {

        If (user.getUsername().equals(username)
            && user.getPassword().equals(password)) {

            System.out.println("\nLogin Successful!");
            System.out.println("Welcome " + user.getUsername());

            Return user;
        }
    }

    System.out.println("Invalid Username or Password");
```



```
        Return null;
    }

}
```

```
import java.util.Scanner;
```

```
public class AdminOperations {
```

```
    private Scanner sc = new Scanner(System.in);
```

```
    // Add Location
```

```
    public void addLocation() {
```

```
        System.out.println("\n===== ADD LOCATION =====");
```

```
        System.out.print("Enter Location ID: ");
```

```
        int id = sc.nextInt();
```

```
        sc.nextLine();
```

```
        System.out.print("Enter Location Name: ");
```

```
        String name = sc.nextLine();
```

```

System.out.print("Enter Address: ");

String address = sc.nextLine();


System.out.print("Enter Seat Capacity: ");

int capacity = sc.nextInt();


Location location = new Location(id, name, address, capacity);


DataStore.locations.add(location);


System.out.println("Location Added Successfully.");
}


// View All Locations
public void viewLocations() {


System.out.println("\n===== LOCATION LIST =====");


if (DataStore.locations.isEmpty()) {
    System.out.println("No Locations Available.");
    return;
}


for (Location location : DataStore.locations) {
    System.out.println(location);
    System.out.println("-----");
}

```

```
}  
}
```

```
// View Pending Events
```

```
public void viewPendingEvents() {
```

```
    System.out.println("\n===== PENDING EVENTS =====");
```

```
    boolean found = false;
```

```
    for (Event event : DataStore.events) {
```

```
        if (event.getStatus().equalsIgnoreCase("Pending")) {
```

```
            System.out.println(event);
```

```
            System.out.println("-----");
```

```
            found = true;
```

```
        }
```

```
    }
```

```
    if (!found) {
```

```
        System.out.println("No Pending Events.");
```

```
    }
```

```
}
```

```
// Approve Event
```

```
public void approveEvent() {
```

```
System.out.print("Enter Event ID to Approve: ");

int eventId = sc.nextInt();

boolean found = false;

for (Event event : DataStore.events) {

    if (event.getEventId() == eventId) {

        event.setStatus("Approved");

        System.out.println("Event Approved Successfully.");

        found = true;
        break;
    }
}

if (!found) {
    System.out.println("Event Not Found.");
}

}

// Monitor Seat Capacity

public void monitorSeatCapacity() {
```

```

        System.out.println("\n===== LOCATION CAPACITY =====");

        for (Location location : DataStore.locations) {

            System.out.println("Location : " + location.getLocationName());
            System.out.println("Capacity : " + location.getCapacity());
            System.out.println("-----");
        }
    }
}

```

```

import java.util.Scanner;

```

```

public class EventOperations {

```

```

    private Scanner sc = new Scanner(System.in);

```

```

    // Create Event

```

```

    public void createEvent(int managerId) {

```

```
System.out.println("\n===== CREATE EVENT =====");
```

```
System.out.print("Enter Event ID: ");
```

```
int eventId = sc.nextInt();
```

```
sc.nextLine();
```

```
System.out.print("Enter Event Name: ");
```

```
String eventName = sc.nextLine();
```

```
System.out.print("Enter Event Date (DD-MM-YYYY): ");
```

```
String eventDate = sc.nextLine();
```

```
System.out.print("Enter Ticket Price: ");
```

```
double ticketPrice = sc.nextDouble();
```

```
System.out.print("Enter Available Seats: ");
```

```
int seats = sc.nextInt();
```

```
if (DataStore.locations.isEmpty()) {
```

```
    System.out.println("No Locations Available. Contact Admin.");
```

```
    return;
```

```
}
```

```
System.out.println("\nAvailable Locations");
```

```
for (Location location : DataStore.locations) {
```

```
        System.out.println(location.getLocationId() + " - " +  
            location.getLocationName());  
    }  
}
```

```
System.out.print("Enter Location ID: ");  
  
int locationId = sc.nextInt();
```

```
Location selectedLocation = null;
```

```
for (Location location : DataStore.locations) {  
  
    if (location.getLocationId() == locationId) {  
        selectedLocation = location;  
        break;  
    }  
}
```

```
if (selectedLocation == null) {  
    System.out.println("Invalid Location.");  
    return;  
}
```

```
Event event = new Event(  
    eventId,  
    eventName,  
    eventDate,
```

```
ticketPrice,  
seats,  
selectedLocation,  
"Pending",  
managerId);
```

```
DataStore.events.add(event);
```

```
System.out.println("Event Created Successfully.");
```

```
System.out.println("Waiting for Admin Approval.");
```

```
}
```

```
// View My Events
```

```
public void viewMyEvents(int managerId) {
```

```
    boolean found = false;
```

```
    System.out.println("\n===== MY EVENTS =====");
```

```
    for (Event event : DataStore.events) {
```

```
        if (event.getManagerId() == managerId) {
```

```
            System.out.println(event);
```

```
            System.out.println("-----");
```

```
            found = true;
```



```

    }
}

if (!found) {
    System.out.println("No Events Found.");
}
}

// Edit Event
public void editEvent(int managerId) {

    System.out.print("Enter Event ID: ");
    int eventId = sc.nextInt();
    sc.nextLine();

    for (Event event : DataStore.events) {

        if (event.getEventId() == eventId
            && event.getManagerId() == managerId) {

            System.out.print("Enter New Event Name: ");
            event.setEventName(sc.nextLine());

            System.out.print("Enter New Ticket Price: ");
            event.setTicketPrice(sc.nextDouble());

```

```
System.out.print("Enter Available Seats: ");  
event.setAvailableSeats(sc.nextInt());
```

```
System.out.println("Event Updated Successfully.");  
return;
```

```
}  
}
```

```
System.out.println("Event Not Found.");  
}
```

```
// Send Event for Approval
```

```
public void sendForApproval(int managerId) {
```

```
System.out.print("Enter Event ID: ");  
int eventId = sc.nextInt();
```

```
for (Event event : DataStore.events) {
```

```
    if (event.getEventId() == eventId  
        && event.getManagerId() == managerId) {
```

```
        event.setStatus("Pending");
```

```
System.out.println("Event Sent for Approval.");  
return;
```

```

    }

}

System.out.println("Event Not Found.");

}

// Cancel Event

public void cancelEvent(int managerId) {

    System.out.print("Enter Event ID: ");

    int eventId = sc.nextInt();

    for (Event event : DataStore.events) {

        if (event.getEventId() == eventId

            && event.getManagerId() == managerId) {

            event.setStatus("Cancelled");

            // Generate coupons for booked consumers

            for (Booking booking : DataStore.bookings) {

                if (booking.getEventId() == eventId) {

                    Coupon coupon = new Coupon(

                        DataStore.coupons.size() + 1,

```

```
        booking.getConsumerId(),  
        "DISC10" + booking.getConsumerId(),  
        10,  
        false);
```

```
        DataStore.coupons.add(coupon);  
    }  
}
```

```
System.out.println("Event Cancelled Successfully.");  
System.out.println("Coupons Generated for Consumers.");
```

```
    return;  
}  
}
```

```
    System.out.println("Event Not Found.");  
}  
}
```

```
import java.util.Scanner;
```

```
public class ConsumerOperations {

    private Scanner sc = new Scanner(System.in);

    // View Approved Events
    public void viewApprovedEvents() {

        System.out.println("\n===== APPROVED EVENTS =====");

        boolean found = false;

        for (Event event : DataStore.events) {

            if (event.getStatus().equalsIgnoreCase("Approved")) {

                System.out.println(event);

                System.out.println("-----");

                found = true;

            }

        }

        if (!found) {

            System.out.println("No Approved Events Available.");

        }

    }

}
```

```
// Book Ticket

public void bookTicket(int consumerId) {

    System.out.print("Enter Event ID: ");

    int eventId = sc.nextInt();

    Event selectedEvent = null;

    for (Event event : DataStore.events) {

        if (event.getEventId() == eventId &&
            event.getStatus().equalsIgnoreCase("Approved")) {

            selectedEvent = event;
            break;
        }
    }

    if (selectedEvent == null) {

        System.out.println("Event Not Found.");

        return;
    }

    System.out.print("Enter Number of Tickets: ");

    int tickets = sc.nextInt();
```

```
if (tickets > selectedEvent.getAvailableSeats()) {  
    System.out.println("Seats Not Available.");  
    return;  
}
```

```
double totalAmount = tickets * selectedEvent.getTicketPrice();
```

```
Booking booking = new Booking(  
    DataStore.bookings.size() + 1,  
    consumerId,  
    eventId,  
    tickets,  
    totalAmount,  
    "Pending");
```

```
DataStore.bookings.add(booking);
```

```
selectedEvent.setAvailableSeats(  
    selectedEvent.getAvailableSeats() - tickets);
```

```
System.out.println("Booking Successful.");
```

```
System.out.println("Amount to Pay : ₹" + totalAmount);
```

```
}
```

```
// Make Payment
```

```
public void makePayment(int consumerId) {
```

```
System.out.print("Enter Booking ID: ");

int bookingId = sc.nextInt();

for (Booking booking : DataStore.bookings) {

    if (booking.getBookingId() == bookingId &&
        booking.getConsumerId() == consumerId) {

        booking.setPaymentStatus("Paid");

        System.out.println("Payment Successful.");
        generateReceipt(booking);

        return;
    }
}

System.out.println("Booking Not Found.");
}

// View My Bookings
public void viewMyBookings(int consumerId) {

    boolean found = false;
```



```

System.out.println("\n===== MY BOOKINGS =====");

for (Booking booking : DataStore.bookings) {

    if (booking.getConsumerId() == consumerId) {

        System.out.println(booking);

        System.out.println("-----");

        found = true;

    }

}

if (!found) {

    System.out.println("No Bookings Found.");

}

}

```

// Apply Coupon

```

public void applyCoupon(int consumerId) {

    sc.nextLine();

    System.out.print("Enter Coupon Code: ");

    String code = sc.nextLine();

    for (Coupon coupon : DataStore.coupons) {

```

```

        if (coupon.getConsumerId() == consumerId &&
            coupon.getCouponCode().equalsIgnoreCase(code) &&
            !coupon.isUsed()) {

            coupon.setUsed(true);

            System.out.println("Coupon Applied Successfully.");
            System.out.println("Discount : "
                + coupon.getDiscountPercentage() + "%");

            return;
        }
    }

    System.out.println("Invalid or Used Coupon.");
}

// Generate Receipt
private void generateReceipt(Booking booking) {

    System.out.println("\n===== E-RECEIPT =====");
    System.out.println("Booking ID : " + booking.getBookingId());
    System.out.println("Consumer ID : " + booking.getConsumerId());
    System.out.println("Event ID : " + booking.getEventId());
    System.out.println("Tickets : " + booking.getNumberOfTickets());
}

```

```

        System.out.println("Amount Paid : ₹" + booking.getTotalAmount());
        System.out.println("Payment Status : " + booking.getPaymentStatus());
        System.out.println("=====");
    }
}

```

```

import java.util.Scanner;

```

```

public class Main {

```

```

    public static void main(String[] args) {

```

```

        Scanner sc = new Scanner(System.in);

```

```

        Authentication auth = new Authentication();

```

```

        AdminOperations adminOps = new AdminOperations();

```

```

        EventOperations eventOps = new EventOperations();

```

```

        ConsumerOperations consumerOps = new ConsumerOperations();

```

```

        while (true) {

```

```

            System.out.println("\n=====");

```

```
System.out.println(" EVENT TICKET BOOKING SYSTEM");  
System.out.println("=====");  
System.out.println("1. Register");  
System.out.println("2. Login");  
System.out.println("3. Exit");  
System.out.print("Enter Choice: ");
```

```
int choice = sc.nextInt();
```

```
switch (choice) {
```

```
    case 1:
```

```
        auth.register();
```

```
        break;
```

```
    case 2:
```

```
        User user = auth.login();
```

```
        if (user == null)
```

```
            break;
```

```
        // ----- ADMIN -----
```

```
        if (user instanceof Admin) {
```

```
while (true) {

    System.out.println("\n===== ADMIN MENU =====");
    System.out.println("1. Add Location");
    System.out.println("2. View Locations");
    System.out.println("3. Approve Event");
    System.out.println("4. View Pending Events");
    System.out.println("5. Monitor Seat Capacity");
    System.out.println("6. Logout");

    System.out.print("Enter Choice: ");
    int adminChoice = sc.nextInt();

    switch (adminChoice) {

        case 1:
            adminOps.addLocation();
            break;

        case 2:
            adminOps.viewLocations();
            break;

        case 3:
            adminOps.approveEvent();
            break;
```

case 4:

adminOps.viewPendingEvents();

break;

case 5:

adminOps.monitorSeatCapacity();

break;

case 6:

System.out.println("Logged Out Successfully.");

break;

default:

System.out.println("Invalid Choice");

}

if (adminChoice == 6)

break;

}

}

// ----- EVENT MANAGER -----

else if (user instanceof EventManager) {

```
while (true) {

    System.out.println("\n===== EVENT MANAGER MENU =====");
    System.out.println("1. Create Event");
    System.out.println("2. View My Events");
    System.out.println("3. Edit Event");
    System.out.println("4. Send Event For Approval");
    System.out.println("5. Cancel Event");
    System.out.println("6. Logout");

    System.out.print("Enter Choice: ");
    int managerChoice = sc.nextInt();

    switch (managerChoice) {

        case 1:
            eventOps.createEvent(user.getUserId());
            break;

        case 2:
            eventOps.viewMyEvents(user.getUserId());
            break;

        case 3:
            eventOps.editEvent(user.getUserId());
```

```
break;
```

```
case 4:
```

```
eventOps.sendForApproval(user.getUserId());
```

```
break;
```

```
case 5:
```

```
eventOps.cancelEvent(user.getUserId());
```

```
break;
```

```
case 6:
```

```
System.out.println("Logged Out Successfully.");
```

```
break;
```

```
default:
```

```
System.out.println("Invalid Choice");
```

```
}
```

```
if (managerChoice == 6)
```

```
break;
```

```
}
```

```
}
```

```
// ----- CONSUMER -----
```



```
else if (user instanceof Consumer) {

    while (true) {

        System.out.println("\n===== CONSUMER MENU =====");
        System.out.println("1. View Approved Events");
        System.out.println("2. Book Ticket");
        System.out.println("3. View My Bookings");
        System.out.println("4. Make Payment");
        System.out.println("5. Apply Coupon");
        System.out.println("6. Logout");

        System.out.print("Enter Choice: ");
        int consumerChoice = sc.nextInt();

        switch (consumerChoice) {

            case 1:
                consumerOps.viewApprovedEvents();
                break;

            case 2:
                consumerOps.bookTicket(user.getUserId());
                break;

            case 3:
```

```
    consumerOps.viewMyBookings(user.getUserId());  
    break;
```

```
case 4:
```

```
    consumerOps.makePayment(user.getUserId());  
    break;
```

```
case 5:
```

```
    consumerOps.applyCoupon(user.getUserId());  
    break;
```

```
case 6:
```

```
    System.out.println("Logged Out Successfully.");  
    break;
```

```
default:
```

```
    System.out.println("Invalid Choice");
```

```
}
```

```
if (consumerChoice == 6)
```

```
    break;
```

```
}
```

```
}
```

```
break;
```

case 3:

System.out.println("Thank You...");

System.exit(0);

default:

System.out.println("Invalid Choice");

}

}

}

}